

Báo cáo rà soát test code

<!-- framework: Jest | run_id: run_20260620_022250_006964 -->

Framework: Jest | Run ID: run_20260620_022250_006964

Tóm tắt kỹ thuật

- Framework: Jest
- Kết quả chạy: Tất cả 11 test được thực thi thành công, coverage 83,33 % (đạt quality gate).
- Vấn đề lớn nhất: Một số test (TC-012, TC-013) cố gắng mock shippingFee nhưng do cách gọi nội bộ trong finalTotal nên mock không có tác dụng, dẫn tới phụ thuộc vào thực thi thực tế của shippingFee và giảm độ tin cậy của test.
- Tài liệu kiểm thử được xem xét: generated.test.js (test) và orderPricing.js (source).

Lỗi nghiêm trọng

- Không phát hiện.

Test còn thiếu

- TC-002 & TC-003: Kiểm thử hàm roundMoney (positive và boundary) được đánh dấu *ambiguous* vì không có export tương ứng trong source. Tuy nhiên roundMoney là hàm nội bộ, không được export, nên không thể test trực tiếp. Cần thêm test gián tiếp thông qua các hàm export (calculateDiscount, finalTotal) hoặc export roundMoney nếu muốn test trực tiếp.
- Kiểm thử shippingFee: Không có test cho các trường hợp:
 - Trọng lượng $\leq 0 \rightarrow$ ném lỗi.
 - Destination không phải "domestic" hoặc "international" $\rightarrow$ ném lỗi.
 - Kiểm thử tính phí cho các mức trọng lượng khác nhau (≤ 5 , $5 < \leq 20$, > 20).
 - Kiểm thử normalizeLabel: Chỉ có một test (TC-001). Cần thêm test cho các giá trị rỗng, null/undefined nếu hàm được gọi với các kiểu không phải string.
 - Kiểm thử finalTotal: Chưa có test cho trường hợp:
 - Subtotal âm (được xử lý trong calculateDiscount, nhưng ảnh hưởng tới finalTotal).
 - order thiếu một trong các thuộc tính bắt buộc (đánh giá lỗi).

Assertion yếu

- TC-012 & TC-013: Mặc dù có jest.spyOn để mock shippingFee, mock không ảnh hưởng vì finalTotal gọi hàm shippingFee nội bộ, không qua đối tượng target. Điều này khiến assertion phụ thuộc vào giá trị thực của shippingFee và không kiểm chứng đúng hành vi mong muốn.
- Bằng chứng: finalTotal trong source: + shippingFee(order.weightKg, order.destination, order.fragile); (gọi trực tiếp).
- Ảnh hưởng: Test không thực sự kiểm tra logic tính tổng khi phí vận chuyển được thay đổi; giảm độ tin cậy và gây rủi ro khi shippingFee thay đổi trong tương lai.

Rủi ro maintainability

- Mock không hiệu quả (TC-012, TC-013) tạo ra "false sense of coverage". Khi shippingFee được sửa đổi, các test này có thể vẫn "pass" mà không phát hiện lỗi.
- Phụ thuộc vào hàm nội bộ: Các test cố gắng mock hàm nội bộ mà không thể, dẫn tới phụ thuộc vào implementation detail, gây khó bảo trì.
- Thiếu test cho các lỗi ngoại lệ (trọng lượng, destination) có thể khiến lỗi mới không được phát hiện khi mở rộng logic.

Gợi ý sửa ngay

1. Sửa cách mock shippingFee

- Thay đổi finalTotal để gọi target.shippingFee (hoặc truyền hàm tính phí như dependency injection) hoặc
- Export shippingFee và trong test dùng jest.spyOn trên module và re-wire hàm nội bộ bằng jest.doMock/jest.mock để thay thế.

2. Thêm test cho shippingFee

- Kiểm tra lỗi khi weightKg ≤ 0 .
- Kiểm tra lỗi khi destination không hợp lệ.
- Kiểm tra các mức phí cho trọng lượng ≤ 5 , $5 < \leq 20$, > 20 , và khi fragile true.

3. Thêm test gián tiếp cho roundMoney

- Sử dụng calculateDiscount hoặc finalTotal để xác nhận việc làm tròn đúng (ví dụ subtotal 10.005, tier standard).

4. Cân nhắc export roundMoney nếu muốn test trực tiếp, hoặc ghi chú trong tài liệu rằng hàm này là internal.

5. Bổ sung test cho normalizeLabel với các giá trị biên (empty string, non-string) để tăng độ bao phủ.

6. Kiểm tra trường hợp thiếu thuộc tính trong order để chắc chắn finalTotal xử lý lỗi một cách rõ ràng (hoặc thêm validation).

Các đề xuất trên sẽ cải thiện độ tin cậy, độ bao phủ và khả năng bảo trì của bộ kiểm thử.